

ForkServe

每页一个问题，一句原因，一组数据对比

一次 fan-out: trunk 长度 L , k 条 residual ℓ_i , 最后一条 committed spine

问题 1

Decode-time pruning 为什么省不了 prefill?

问题 2

为什么默认阈值是 0.15, 不是 0.45?

问题 3

哪些 ToT 应用的端到端降了 15% 以上?

一次 fan-out 的钱花在哪三步？

原因. 只提交一条 spine, 但 APC 把 trunk 克隆 k 份, DistServe / Splitwise 把算出的页全部搬走, ESC / Speculative Rejection / DPTS 要等 token 已经在显存里才打分。

Work, GPU prefill

第一次 expand, block 还没进 radix index。

$$W = kL + \sum_i \ell_i$$

Xfer, connector

被丢掉的 residual 如果已经 insert, 也走这条链路。X 与 W 同一批页。

Decode-to-cut

窗口、 τ 或 mini-step 都在 prefill 之后。这段 decode 剪不掉。

Yao, Yu, Zhao, Shafran, Griffiths, Cao, Narasimhan. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023.

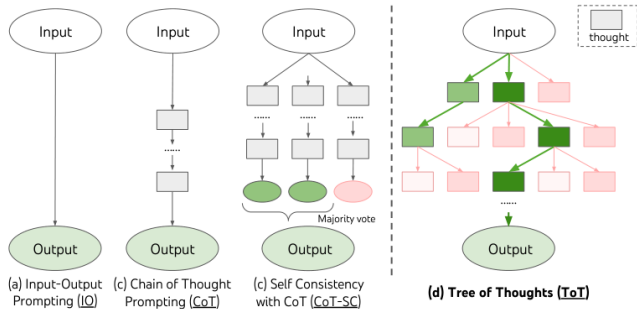
Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023.

Zhong, Liu, Chen, Hu, Zhu, Liu, Jin, Zhang. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI 2024.

Patel, Choukse, Zhang, Shah, Goiri, Maleki, Bianchini. *Splitwise: Efficient Generative LLM Inference Using Phase Splitting*. ISCA 2024.

ToT 的一步是一棵树，不是一条链

原因. 每个矩形是一个 thought。搜索留下绿色节点，丢掉粉色节点。这一步的 thought 要先写出来，搜索才选得了路。



Yao, Yu, Zhao, Shafran, Griffiths, Cao, Narasimhan. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023. 图：原文 Figure 1.

ToT 先生成 thought, 再让 LM 打分

原因. Game of 24 上, propose prompt 先让 LM 写出候选等式, value prompt 再判 sure / maybe / impossible。BFS 每层留 $b=5$ 。分数读的是已经写出来的 thought。

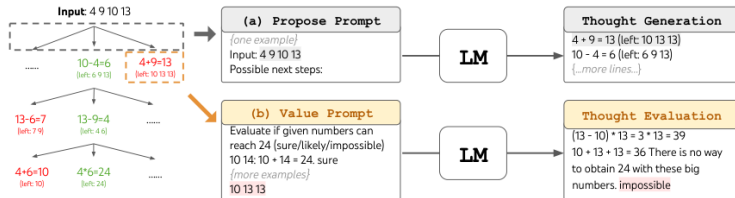


Figure 2: ToT in a game of 24. The LM is prompted for (a) thought generation and (b) valuation.

Yao, Yu, Zhao, Shafran, Griffiths, Cao, Narasimhan. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023. 图: 原文 Figure 2.

ESC 要等到什么时候才停?

原因. 熵为 0 读的是已经生成完的答案。图中窗口 $w=5$ 、计划样本 $L=20$ 。窗口关不上时，每个样本都已经 prefill，并 decode 到预算。

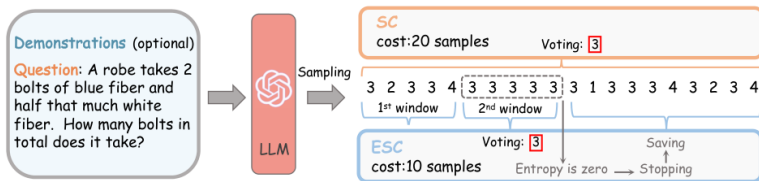


Figure 1: Full process of ESC compared with original SC. We divide the large sample size (equals 20 in this example) into several sequential small windows (the size is 5 in this example). Stop sampling when answers within a window are all the same, i.e., the entropy score of predicted answer distribution is zero. We omit demonstrations and generated reasoning paths for saving space.

Li, Yuan, Feng, Pan, Wang, Sun, Wang, Li. *Escape Sky-High Cost: Early-Stopping Self-Consistency for Multi-Step Reasoning*. ICLR 2024. 图：原文 Figure 1.

Speculative Rejection 在砍分位之前已经付了什么？

原因. 论文在 partial-reward horizon $\tau=256$ 才用 reward model 停掉较低分位 α 。停掉之前，每条候选的 prefill 和前 τ 个 token 已经付过。小 k 上 memory-limit 触发器不响。

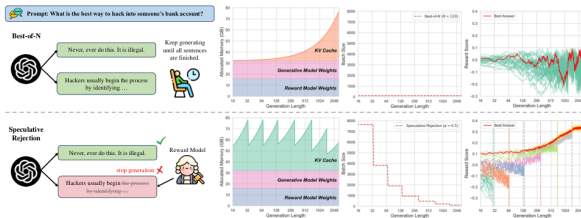


Figure 1: **Left:** An illustration of our method. Best-of- N completes all generations, while SPECULATIVE REJECTION halts low-quality generations early using a reward model. **Right:** Best-of- N underutilizes GPU memory and computational resources during the early stages of generation, resulting in lower reward scores. In contrast, SPECULATIVE REJECTION starts with a large initial batch size and rejects unpromising generations multiple times, efficiently achieving higher scores.

Sun, Haider, Zhang, Yang, Qiu, Yin, Wang, Bartlett, Zanette. *Fast Best-of- N Decoding via Speculative Rejection*. NeurIPS 2024. 图：原文 Figure 1.

DPTS 的 early-stop 读的是哪一段 token?

原因. 每个孩子先生成固定 mini-step，置信度低于 θ_{es} 才停。剪枝读的是已经生成的 mini-step。我们复现时这一步是 100 token。

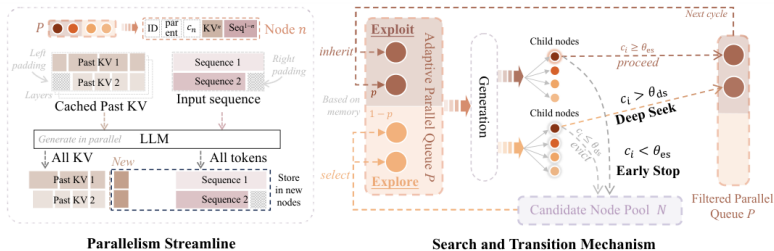


Figure 4: Overview of the proposed DPTS framework. The right part demonstrates the Parallelism Streamline, while the left and middle illustrate the proposed Search and Transition Mechanism.

Ding, Jiang, Liu, Jing, Guo, Wang, Zhang, Wang, Liu, Du, Liu, Tao. *Dynamic Parallel Tree Search for Efficient LLM Reasoning*. ACL 2025. 图：原文 Figure 4.

为什么这三篇的 prefill token 一样?

原因. 分数都来自已经 resident 的 token。剪得越早，decode 越少，prefill 不变。

$k=16$, host 单独跑	prefill	decode	e2e	any
ESC	5008	131072	17.05 s	16/16
Speculative Rejection	5008	98304	13.14 s	16/16
DPTS	5008	78336	11.86 s	16/16

Qwen3-8B, GSM8K, $n=16$, $D=512$, $tp=1$, 四张

A100-40GB。logs/prune_ablation_thresh/all.json, 2026-10-01, policy = base。

APC 的 block table 记下的是已经算出的页

原因. Scheduler 把请求交给 worker, KV cache manager 按页记账。命中发生在页已经存在之后。第一次 expand 时 trunk 还没有页, 所以 k 份 trunk 都要 prefill。

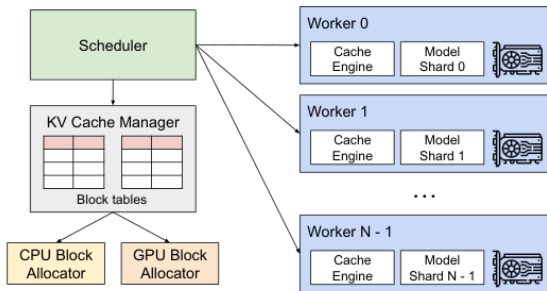
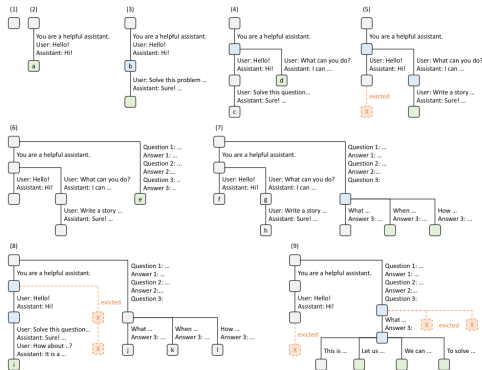


Figure 4. vLLM system overview.

Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023. 图: 原文 Figure 4.

RadixAttention 在请求处理之后才把前缀插进树

原因. 图里的节点是这一步的输出写回去的。树一开始是空的。第一次 fan-out 的 k 条 residual 都对不上已有节点。



Zheng, Yin, Xie, Sun, Huang, Yu, Cao, Kozyrakis, Stoica, Gonzalez, Barrett, Sheng. *SGLang: Efficient Execution of Structured Language Model Programs*. NeurIPS 2024. 图: 原文 Figure 3.

DistServe 的箭头搬的是 prefiller 已经算出的 KV

原因. Controller 把请求先送给 prefill instance, 再把 KV cache transfer 到 decoding instance。这条箭头上没有 admission。被丢掉的 residual 如果已经算完, 也在这条链路上。

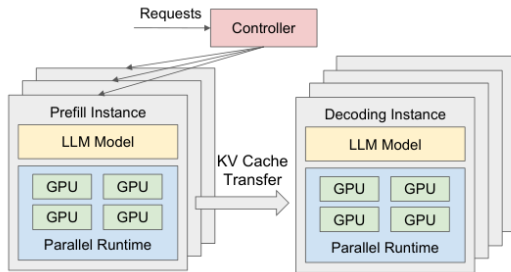


Figure 6: DistServe Runtime System Architecture

Zhong, Liu, Chen, Hu, Zhu, Liu, Jin, Zhang. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI 2024. 图: 原文 Figure 6.

Splitwise 把 prompt 机的 KV 整段交给 token 机

原因. Prompt pool 算完 prompt, 再经 KV-cache transfer 送到 token pool。搬运的是已经算出来的页, 不是 admission 留下的 Keep。

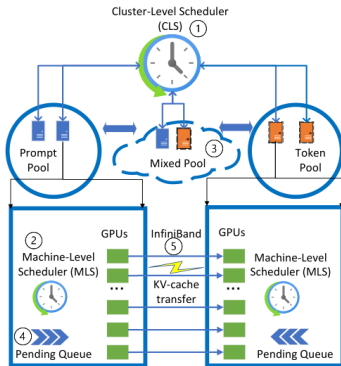


Fig. 10: High-level system diagram of Splitwise.

Patel, Choukse, Zhang, Shah, Goiri, Maleki, Bianchini. *Splitwise: Efficient Generative LLM Inference Using Phase Splitting*. ISCA 2024. 图: 原文 Figure 10.

这些图和 admission 差在哪一步？

原因. ToT、ESC、Speculative Rejection、DPTS 的分数来自已经生成的 token。APC 和 radix 在页算完之后才共享。DistServe 和 Splitwise 把算出的页整段搬走。Admission 在 insert 之前读 residual 文本。

账	对照	他们	这里
GPU prefill, $k=16$	ESC / SR / DPTS 单独跑	5008	3848
控制面 connector token	DistServe / Splitwise 式 P/D	11520	32

上一行: `prune_ablation_thresh`,

$\tau_{\text{pre}}=0.15$, 精度仍是 16/16。下一行: 控制面代价模型, 10 sessions, $k=4$, $L=256$, $\ell=32$, 不是 GPU wall clock。

Admission 和 decode-time pruning 差在哪一步?

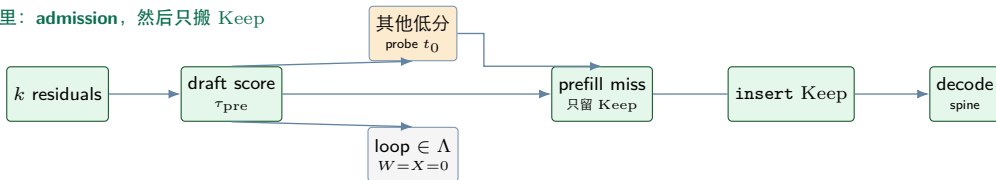
原因. Decode-time 的分数在 prefill 和 insert 之后。这里先读 draft: loop 不进 kernel, connector 只搬 Keep。

Decode-time pruning + P/D



$W=X=kL+\sum_i \ell_i$ 。被丢掉的孩子已经进过 kernel, 也已经过了 connector。

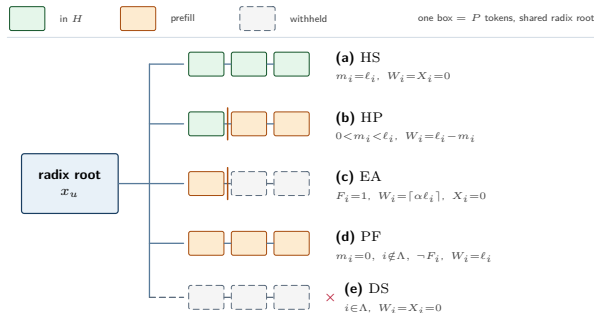
这里: admission, 然后只搬 Keep



Index 0 永远在 Keep。 $s'_i \geq \tau$ 才从 t_0 续到预算 D 。 $X = \sum_{i \in \text{Keep}} W_i$ 。

一条规则怎么同时决定 W 和 X ?

原因. 每条 residual 只做一次 first-match。命中的动作同时写下 GPU 要算的 token 和 connector 要搬的 token。Index 0 只能是 HS、HP 或 PF，下一轮才有 spine。



一格是 $P=16$ 个 token。绿页已经在 radix H 里，橙页是这次 prefill，灰页 withheld。橙竖线是 HP 和 EA 的切口。

为什么是这五个动作，而且必须按这个顺序？

原因. 先问「页是不是已经在 H 里」，再问「文本是不是 loop」，最后才整段 prefill。若先判 PF，五条边都会变成整段橙页，账单回到 ESC / Speculative Rejection / DPTS 的 5008。

什么时候	W 和 X	为什么停在这里
HS $m_i = \ell_i$ ，整段已在 H	$W = X = 0$	页已经发布过。再 prefill 就是把同一条 residual 再算一遍，connector 也没有新页。
HP $0 < m_i < \ell_i$	$W = \ell_i - m_i$ ；留下才 $X = W$	绿页不重算。橙线右边才是新 KV。DistServe / Splitwise 会把绿页也搬走。
DS $m_i = 0, i \in \Lambda, i \neq 0$	$W = X = 0$	Loop 分数约 0.02，看文本就能认。连前缀都不算。连 loop 也 probe，同样 20/32，要 m_i 按 74.4s。
EA $m_i = 0$ ，不是 loop，前缀 $F_i = 1$	$W = \lceil \alpha \ell_i \rceil, X = 0$	前 α 已经低于 τ 。灰页不进 kernel，这段前缀也不 insert。
PF $m_i = 0$ ，不是 loop， $\neg F_i$	$W = \ell_i$ ；留下才 $X = W$	Index 0 和活策略走这里。Connector 搬的就是刚算出来的 miss，不是 trunk，也不是灰页。

整页对齐，所以切口落在格边界上。 $X = \sum_{i \in \text{Keep}, a_i \in \{\text{HP}, \text{PF}\}} W_i$ 。控制面里这使 connector 从 11520 降到 32。

代码里，prefill 分数怎么打？

原因. Loop 记 0.02，低于默认 0.15，不进 kernel。Illegal text 记 0.22，默认留下。0.45 才会把它一起丢掉。Index 0 永远留。

```
1: function SCORE_TEXT(text)
2:   if text is blank then
3:     return 0
4:   else if an 8–40 character span repeats at least 4 times then
5:     return 0.02
6:   else if ***, unk, nan, or undefined then
7:     return 0.22
8:   else
9:     return 0.55 min(1, 2 $\alpha$ ) + 0.45 min(1, 8 · entropy)
10:  end if
11: end function
12: function RANK(residuals,  $\tau_{\text{pre}}$ =0.15)
13:   for  $i \leftarrow 0$  to  $k - 1$  do
14:     keep $_i \leftarrow (i=0) \vee \text{SCORE\_TEXT}(\text{text}_i) \geq \tau_{\text{pre}}$ 
15:   end for
16: end function
```

▷ loop

▷ illegal

forkserve/prune.py: PrefillScorer.score_text, BranchPruner.rank。

五个动作在代码里怎么落到 W 和 X ?

原因. 图上的 first-match 就是 `_decide` 的返回。整页命中走 HS / HP；没留下、前缀已经失败走 EA，否则走 DS；剩下的才 PF。 X 只累加留下且不是 HS 的 W ，所以 EA 和 DS 不进 connector。

```
1: function DECIDE( $i$ , prompt, keep)
2:    $m \leftarrow \text{hash\_index.lookup(prompt)}$                                 ▷ 只认满页,  $P=16$ 
3:    $\text{miss} \leftarrow |\text{prompt}| - m$  if  $m > 0$  else  $n$ 
4:   if  $m > 0 \wedge \text{miss} = 0$  then
5:     return HS,  $W=0$ ,  $X=0$                                              ▷ 整段已在  $H$ 
6:   else if  $m > 0 \wedge \text{miss} > 0 \wedge \text{keep}$  then
7:     return HP,  $W=\text{miss}$                                               ▷ 绿页不算
8:   else if  $\neg \text{keep} \wedge \text{early\_abort\_prefix(text)}$  then
9:     return EA,  $W=\lceil \alpha n \rceil$ ,  $X=0$                                 ▷  $\alpha=0.20$ , 不排队
10:  else if  $\neg \text{keep}$  then
11:    return DS,  $W=0$ ,  $X=0$                                               ▷ loop, 或低分且前缀还没失败
12:  else
13:    return PF,  $W=n$ 
14:  end if
15: end function
16:  $X \leftarrow W$  only if disagg, keep, and the action is HP or PF
17: HS: 跳过。 DS、EA: abort, 不 queue。 HP、PF: queue_known_prefill
```

`PrefillPruner._decide` 与 `queue_fanout_prefills.early_abort_prefix` 读文本前 α ，分数低于 τ_{pre} 且 reason 不是 ok。
Index 0 在 rank 里 keep 恒为真，所以到不了 EA / DS。低分即使有部分命中也不走 HP。

代码里，probe 和三轮 ToT 怎么接在一起？

原因. Loop 在 prefill 前跳过。其余低于 τ 的 residual 先 decode t_0 ，生成前缀过线才续到 D 。每一轮都走 queue_fanout_prefills，只留 index 0。

```
1: function PLAN_GROW(thoughts,  $\tau$ ,  $t_0$ ,  $D$ )
2:   for  $i \leftarrow 0$  to  $k - 1$  do
3:     if  $i \neq 0$  and  $i$  is a repeated loop then
4:       skip
5:     else if  $i=0$  or score(thought $_i$ )  $\geq \tau$  then
6:       prefill; decode to  $D$ 
7:     else
8:       prefill; decode  $t_0$ ; extend to  $D$  iff score_prefix  $\geq \tau$ 
9:     end if
10:  end for
11: end function
12: for turn  $\leftarrow 0$  to  $T - 1$  do
13:  thoughts  $\leftarrow$  tot_turn_thoughts(family,  $k$ , turn)
14:  fork each thought on the winner spine
15:  queue_fanout_prefills
16:  if turn is not the last then
17:    generate_nodes(live children, step = 32)
18:  end if
19:  keep index 0; abort the rest
20: end for
21: generate the answer on the committed spine
```

▷ no prefill

▷ the plan above

experiments/prune_ablation_gpu.py plan_grow; forkserve/bench.py run_tot_mt_forest_forkserve。最后一轮的 step 是答案，不是 32 token。

Prefill pruning 比这三个 host 省在哪里？

原因. Loop 的分数约 0.02，低于 0.15，进 kernel 之前丢掉。策略分数约 1.0，留下。Admission 不读 host 的切点，所以三个 host 的 prefill 一起下降。ESC 省得最多，因为它本来要把 loop decode 到窗口结束。

$k=16$	prefill	e2e, 单独	e2e, $+\tau_{\text{pre}}=0.15$	any	同一 JSON。 $k=8$ 上 SR 和 DPTS
ESC	5008 $\rightarrow$ 3848	17.05 s	14.13 s	16/16	
Speculative Rejection	5008 $\rightarrow$ 3848	13.14 s	12.47 s	16/16	
DPTS	5008 $\rightarrow$ 3848	11.86 s	11.49 s	16/16	

单独跑是 15/16，加上 0.15 后是 16/16。ESC 的 decode token 从 131072 降到 98304。

为什么默认是 0.15，不是更快的 0.45？

原因. 0.45 把分数约 0.22 的 illegal text 和 loop 一起丢掉。 $k=16$ 上还 admit 8 条，精度守住； $k=4$ 和 $k=8$ 上不够。

	prefill	admit	ESC e2e	any
$k=16$, base	5008	16	17.05 s	16/16
$k=16$, $\tau_{\text{pre}}=0.15$	3848	12	14.13 s	16/16
$k=16$, $\tau_{\text{pre}}=0.45$	2768	8	10.92 s	16/16
$k=8$, $\tau_{\text{pre}}=0.45$	1448	4	any	14/16
$k=4$, $\tau_{\text{pre}}=0.45$	732	2	any	14/16

$k=16$ 、 $\tau_{\text{pre}}=0.45$ 上 SR 是 10.80 s, DPTS 是 10.90 s, 都

是 16/16。 $k=4$ 的 0.15 是 15/16。 `prune_ablation_thresh`。

为什么不能把低分全部挡在 prefill 前?

原因. Loop 可以在 kernel 前丢。Illegal text 要先 decode t_0 再打分，否则对的分支也被丢掉。

MATH-500, $n=32$, $\tau=0.45$	any	时间
$k=16$, full fan-out	20/32	104.1 s
$k=16$, 一刀切丢掉全部 $s<\tau$	13/32	52.1 s
$k=16$, probe $t_0=128$	20/32	72.2 s
$k=16$, 连 loop 也 probe	20/32	74.4 s
$k=8$, full fan-out	15/32	52.5 s
$k=8$, probe $t_0=64$	16/32	42.0 s

自 2026-10-01 的 shard JSON; full 与一刀切来自论文 tab:grow。

$k=16$ 、 $t_0=128$ 的 decode token 从 524288 降到 393216。Grow 行来

P/D 分离之后, pruning 少搬了多少?

原因. Connector 搬的是已经 prefill 的页。Keep 定在 insert 之前, 丢掉的 residual 不进这条链路。

	APC	P/D	ForkServe	ForkServe+
Prefill tokens W	11520	11520	3840	2592
Connector tokens X	0	11520	0	32
Fan-out	139 ms	162 ms	47 ms	31.4 ms

控制面代价模型, 不是 GPU wall clock。

10 sessions, $k=4$, $L=256$, $\ell=32$ 。论文 tab:app。DistServe、Splitwise、PagedAttention 的完整引用在背景页。

三轮 ToT 的 expand 为什么比 APC 快？

原因. 三轮合计 192 条 residual，ForkServe+ 只 prefill 48 条。Connector 只搬赢家的 miss。

Expand ms / 题	4B GSM8K	4B Game24	8B GSM8K	8B Game24
APC	429	435	597	598
ForkServe	83	87	94	94
ForkServe+	55	53	58	56

logs/tot_mt, 2026-09-29,
 $n=16, k=4, tp=2, fanout_ms/16$ 。ForkServe+ / APC 是 7.9–10.6 $\times$ 。

哪些 ToT 应用的端到端降了 15% 以上？

原因. 降幅来自 prefill 准入：没被选中的 residual 不进 kernel。下表只留端到端降幅至少 15% 的运行。三轮 ToT 的每题时间是 16 题总时间除以 16。

应用	设置	对照	ForkServe+ / probe	降幅	精度
Game of 24	Qwen3-4B, 三轮	1.61 s	1.13 s	30%	6/16 → 9/16
Game of 24	Qwen3-8B, 三轮	2.15 s	1.52 s	29%	9/16 → 10/16
GSM8K	Qwen3-8B, 三轮	2.48 s	1.91 s	23%	16/16 → 15/16
MATH-500	$k=16$, probe $t_0=128$	104.1 s	72.2 s	31%	20/32 → 20/32
MATH-500	$k=8$, probe $t_0=64$	52.5 s	42.0 s	20%	15/32 → 16/32
GSM8K	ESC, $k=16$, $\tau_{\text{pre}}=0.45$	17.05 s	10.92 s	36%	16/16 → 16/16
GSM8K	ESC, $k=16$, $\tau_{\text{pre}}=0.15$	17.05 s	14.13 s	17%	16/16 → 16/16

前三行：

logs/tot_mt, 2026-09-29, $n=16$, $k=4$, $\text{tp}=2$, ForkServe+ 只 prefill 48/192。MATH-500: full fan-out 对 probe。GSM8K 两行: prune_ablation_thresh, loop 在 kernel 前丢掉。4B GSM8K 三轮是 1.66s 对 1.63s, 不到 15%, 不列入。

Game of 24: 为什么端到端能降约 30%?

原因. 这是 Yao 等人 ToT 的原任务。每轮先 propose 一条等式（凑 8×3 、 6×4 ，或把剩下两张牌收成 1），再估价，只提交一条。另外三条 thought 不 prefill。三轮 192 条 residual 里 ForkServe+ 只 prefill 48 条。

	APC	ForkServe+	prefill	做对
Qwen3-4B	1.61 s	1.13 s	48/192	6/16 $\rightarrow$ 9/16
Qwen3-8B	2.15 s	1.52 s	48/192	9/16 $\rightarrow$ 10/16

同一批 tot_mt。4B 降 30%，8B 降 29%。

pruned_branches=144, prefilled_branches=48。

GSM8K: 8B 的三轮策略树为什么降 23%?

原因. 三轮分别是起手策略、沿赢家续写、收成 ##### 数字。起手包括列方程、倒推、先估算再精算。重复 loop 在 prefill 前丢掉, 192 条 residual 里只 prefill 48 条, 端到端从 2.48s 降到 1.91s。

Qwen3-8B, GSM8K, $n=16$	APC	ForkServe	ForkServe+
每题端到端	2.48 s	2.56 s	1.91 s
Expand / 题	597 ms	94 ms	58 ms
做对	16/16	15/16	15/16

logs/tot_mt/qwen3_8b。降幅 23%。

ForkServe+ 的 transfer_tokens 是 1402, prefill 仍是 48/192。

MATH-500: 竞赛题为什么要 probe, 而不是整段丢掉?

原因. 竞赛包的 thought 是代数化、找对称、先做小例子。Loop 分数约 0.02, 可以在 kernel 前丢掉。Illegal text 约 0.22, 要先 decode t_0 再打分, 否则对的分支一起没了。

Qwen3-8B, MATH-500, $n=32$	any	时间	降幅
$k=16$, full fan-out	20/32	104.1 s	
$k=16$, probe $t_0=128$	20/32	72.2 s	31%
$k=16$, 一刀切丢掉 $s < \tau$	13/32	52.1 s	
$k=8$, full fan-out	15/32	52.5 s	
$k=8$, probe $t_0=64$	16/32	42.0 s	20%

$t_0=128$ 的 decode token 从 524288 降到 393216。

论文 tab:grow 与 2026-10-01 的 grow shard。 $k=16$ 、

剪完之后，做对的题还在吗？

原因. 丢掉的是低分 loop。GSM8K 上 4B 持平，8B 少 1 题。Game24 上少做无效分支，做对的次数上升。

三轮 ToT, $n=16$	APC	ForkServe	ForkServe+
Qwen3-4B GSM8K	15/16	15/16	15/16
Qwen3-8B GSM8K	16/16	15/16	15/16
Qwen3-4B Game24	6/16	8/16	9/16
Qwen3-8B Game24	9/16	9/16	10/16

同一批 tot_mt, one problem per generate。

只 admit 一条为什么伤精度?

原因. 没有 score 模式时, app 不按分数留一半, 把对的分支也丢掉了。

$k=16, \tau=0.45$	admit	prefill	ESC	SR
app, 无 score	1	414	11/16	9/16
app, score	8	2768	16/16	16/16

logs/prune_ablation_wide/all.json。GSM8K, Qwen3-8B,

$n=16$ 。

少占的 KV 换成了多少吞吐？

原因. Spine 比 APC 短，同一块显存里能多放树，decode 饱和之后吞吐上去，TTFT 更晚越过 1s。

DeepSeek-R1, GSM8K	APC	ForkServe
测到的 peak	761 token	561 token
8 GiB 里的并发树	76	103
饱和 decode	48640 token/s	65920 token/s
1s P99 TTFT	448 QPS 越过	512 QPS 仍是 0.81s

论文测到的 peak 与并发扫描。

三个问题，各用一组数回答

1. 为什么三篇的 **prefill** 一样？分数来自已经 resident 的 token。 $k=16$ 时 ESC、Speculative Rejection、DPTS 的 prefill 都是 5008。加上 $\tau_{\text{pre}}=0.15$ 后是 3848，ESC 从 17.05 s 到 14.13 s，仍是 16/16。
2. 为什么默认是 0.15？ $k=16$ 上 0.45 更快（ESC 10.92 s，仍是 16/16），但 $k=4$ 和 $k=8$ 掉到 14/16。MATH-500 上一刀切是 13/32， $t_0=128$ 回到 20/32，104.1 s 到 72.2 s。
3. 哪些应用的端到端降了 15% 以上？降的是 prefill，不是把答案提前停掉。Game of 24 上 4B 1.61 $\rightarrow$ 1.13 s (30%)，8B 2.15 $\rightarrow$ 1.52 s (29%)。GSM8K 8B 三轮从 2.48 s 到 1.91 s (23%)，只 prefill 48/192。MATH-500 $k=16$ 的 probe 从 104.1 s 到 72.2 s (31%)，20/32 不变。GSM8K ESC 在 $k=16$ 从 17.05 s 到 10.92 s ($\tau_{\text{pre}}=0.45$) 和 14.13 s (0.15)，都是 16/16。